CSE PROJECT VIVA & TECHNICAL INTERVIEW HANDBOOK

University Assignment Portal

MERN Stack Project — Complete Study Guide

What is it? · How does it work in MY project? · Why did I use it?
What will the professor ask? · What should I answer? · What comes next?

Prepared for revision before the viva — read once, remember forever.

TABLE OF CONTENTS

1. **Project Overview — What, Why, Who, Workflow**
2. **Technology Stack — Tool by Tool**
3. **JWT & Authentication Workflow**
4. **Database & Schema Design**
5. **Routes & API Request Flow**
6. **Cloudinary & File Upload (Crucial Topic)**
7. **Email System — Nodemailer**
8. **Frontend — React**
9. **Security Checklist**
10. **Error Handling & Deployment**
11. **Tricky Professor Follow-Up Questions**
12. **Top 30 Must-Memorize Viva Questions**
13. **2-Minute & 5-Minute Project Explanations**

HOW TO USE THIS HANDBOOK

Every topic answers 5 things: **What is it?** → **How does it work in MY project?** → **Why did I use it?** → **Likely question** → **Ideal answer**. Orange boxes are follow-up questions your professor may ask next — practice answering those out loud.

1 · PROJECT OVERVIEW

The big picture — what, why, and who

What: What is the project?

A: The University Assignment Portal is a web application that helps universities manage student assignment submissions digitally, replacing scattered email/WhatsApp submissions with one organized system.

Why: What problem does it solve, and why was it created?

A: It solves the problem of professors getting overwhelmed by assignment files scattered across email and WhatsApp. It was created to keep academic submissions organized in one place, with dedicated portals for students, professors, and administrators.

Who are the users?

Admin	Sets up departments and creates student/professor accounts.
Student	Selects their professor and uploads PDF assignment files.
Professor	Views submitted assignments, reviews PDFs on the portal, and approves or rejects them.

Complete end-to-end workflow

1. Admin creates a user account (student or professor).
2. System auto-generates a password and emails it via Nodemailer.
3. User logs in with the emailed credentials.
4. System forces a password change on first login (mustChangePassword flag).
5. Student uploads a PDF assignment.
6. PDF is saved to Cloudinary; only the URL is stored in MongoDB.
7. Professor reviews the PDF directly inside the portal.
8. Professor approves or rejects the assignment.

Technology stack at a glance

Frontend	React, Vite, React Router, TailwindCSS/UI, Axios, Socket.io-client
Backend	Node.js, Express, Socket.io
Database	MongoDB with Mongoose
Authentication	JWT (HttpOnly cookies) + BcryptJS

File Storage	Cloudinary (via Multer)
Email	Nodemailer

Architecture in one sentence

Explain: Explain the complete architecture simply.

A: This is a Client-Server architecture. The React frontend runs in the browser and sends HTTP requests via Axios to the Express backend. The backend talks to MongoDB (for data) and Cloudinary (for files), then returns a JSON response that updates the screen. Flow: **User** → **React Frontend** → **Axios (API call)** → **Express Backend** → **MongoDB / Cloudinary** → **JSON response** → **React updates UI.**

2 · TECHNOLOGY STACK

Tool by tool — where, why, and how to explain it

Technology	Where Used	Why Used	Simple Viva Answer
React & Vite	Frontend architecture	React builds interactive UIs from reusable components. Vite makes development and reloading extremely fast.	"I used React to build a fast single-page app where pages load without refreshing. Vite made development much faster."
Node.js & Express	Backend server	Node.js runs JavaScript on the backend. Express is a framework that makes it easy to create API routes and handle requests.	"I used Node.js and Express to create my backend API, which connects React to MongoDB."
MongoDB & Mongoose	Database	MongoDB stores data as JSON-like documents, matching JavaScript naturally. Mongoose adds schemas to keep data organized.	"I used MongoDB to store user and assignment data because it's flexible and works seamlessly with Node.js."
JWT	Authentication system	Keeps users logged in securely without storing session data on the server (stateless).	"After login the backend issues a token, stored in a secure cookie, which proves identity on every request."
Cloudinary	File and image management	Storing large PDFs directly in MongoDB is inefficient and slows the database down.	"I used Cloudinary to store PDF files, keeping my database lightweight — it only stores the file URL."
Nodemailer	Backend utility	Used to send emails automatically from the server.	"I used Nodemailer to send welcome emails with auto-generated passwords, and OTPs for password resets."
Socket.io	Frontend & backend	Enables real-time updates without refreshing the page.	"I used Socket.io for real-time chat and to show who is currently online."

3 · JWT & AUTHENTICATION WORKFLOW

The #1 topic professors ask about

Q: How does JWT authentication work in your project?

A: On login, the backend verifies email and password. If correct, it generates a JWT containing the user's ID and role, and stores it inside a secure HttpOnly cookie (not the JSON body). On every future request, Axios (with withCredentials: true) auto-attaches the cookie. Middleware such as verifyStudent/verifyProfessor reads the cookie, verifies the token, and allows access to protected routes.

Q: Where is the token stored, and why?

A: In an HttpOnly cookie — this is safer than localStorage because JavaScript (and XSS attacks) cannot read HttpOnly cookies.

Q: How are passwords stored?

A: Never as plain text. A Mongoose pre('save') hook hashes the password with bcryptjs (salt factor 10) before saving. On login, bcrypt.compare() checks the entered password against the stored hash.

Q: How does the auto-generated password system work?

A: If an Admin doesn't set a password, the backend generates an 8-character random string with Math.random() and sets mustChangePassword: true. On first login the server detects this flag and the frontend redirects the user to a Force-Change-Password page before the dashboard unlocks.

Q: How is logout handled?

A: The frontend calls POST /logout; the backend clears the HttpOnly cookie holding the JWT, destroying the session.

Follow-up the professor will almost certainly ask

Professor asks: *If the JWT is stored on the client side, how do you prevent hackers from stealing it using JavaScript (XSS)?*

Ideal answer: The JWT is NOT stored in localStorage where JavaScript can read it. The backend sends it back as an HttpOnly cookie — this tells the browser to lock the cookie away from JavaScript entirely. The browser only attaches it automatically to API requests.

4 · DATABASE & SCHEMA DESIGN

Q: Why did you choose MongoDB?

A: MongoDB stores data as documents that look like JSON. Since React and Node.js both use JavaScript/JSON natively, saving and transferring data felt very natural and fast.

How the schemas are designed

UserData	Stores Students, Professors, and HODs. Has a **role** field to distinguish them, and an ObjectId reference to their Department.
Admin	A separate model just for administrators, for added security.
Assignment	Stores submission details: Student ObjectId, Professor ObjectId, Cloudinary file URL, and status (Draft, Submitted, Approved, Rejected).

Q: Why did you use ObjectId references instead of duplicating data?

A: Instead of copying a professor's name/email into every assignment, I store their unique ObjectId — keeping the database normalized. When professor details are needed, Mongoose's .populate() fetches the related data automatically.

5 · ROUTES & API REQUEST FLOW

How does a request move through the backend?

1. Axios (frontend) sends a request, e.g. POST /api/student/assignments/upload.
2. It hits the Router (studentRoutes.js).
3. Authentication Middleware reads the JWT cookie to confirm the student is logged in.
4. Multer Middleware temporarily saves the uploaded PDF to the server's disk.
5. The Controller (assignmentController.js) uploads the PDF to Cloudinary, deletes the temp file, saves the URL to MongoDB, and returns a JSON success response.

HTTP methods used and why

GET	Retrieve data — e.g. list of professors, dashboard stats.
POST	Create new data or send secure data — e.g. submitting an assignment, logging in.
PUT / PATCH	Update existing data — e.g. professor changes assignment status to Approved.
DELETE	Remove data from the database.

6 · CLOUDINARY & FILE UPLOAD

Crucial topic — expect deep questioning here

How does PDF upload work, step by step?

1. Student selects a PDF file (under 10MB).
2. React frontend sends it via FormData using Axios.
3. Backend uses Multer to temporarily save the file to the server's tmp folder (os.tmpdir()).
4. Backend calls cloudinary.uploader.upload() with resource_type 'raw' (since it's a PDF, not an image).
5. Cloudinary returns a secure URL for the file.
6. Backend saves that URL in the MongoDB Assignment document.
7. Crucial step: backend runs fs.unlinkSync() to delete the temporary local file, so server storage never fills up.

Q: How does the Professor view the file?

A: The React frontend gets the Cloudinary URL from the database and embeds it directly into an <iframe>, so the professor can read the PDF right on the webpage without downloading it.

Professor asks: *Why did you use Cloudinary? Why not just save the PDF directly into MongoDB?*

Ideal answer: MongoDB has a strict 16MB document size limit. Storing PDFs as binary data would bloat the database, slow down search queries, and risk hitting that limit. Cloudinary handles large files efficiently, acts as a CDN for fast loading, and keeps the database lightweight by storing only the URL string.

7 · EMAIL SYSTEM — NODEMAILER

Q: How does your system send emails?

A: Using the Nodemailer library with SMTP credentials (SMTP_USER, SMTP_PASS) stored safely in a .env file. When an admin creates a new user, the backend builds an HTML template with the login details and auto-generated password, and Nodemailer dispatches it automatically. The same system sends a 4-digit OTP during the Forgot Password flow, storing it in the database with a 15-minute expiry (resetPasswordExpires).

8 · FRONTEND — REACT

Q: How do you handle state in React?

A: Mainly with React Hooks — useState for local component state (like form inputs), and useEffect for fetching data from the API as soon as a component mounts.

Q: How are protected pages handled?

A: Layout components (StudentLayout, AdminLayout) ping /api/auth/me when they load. If the backend rejects the request (invalid JWT), the user is redirected to login. The JWT itself is never stored in localStorage — only basic non-sensitive profile info — because the backend reads the secure cookie directly.

Q: How are errors displayed to the user?

A: try/catch around Axios calls. On error, react-hot-toast shows a popup with the message sent by the backend controller, e.g. toast.error(err.response.data.message).

9 · SECURITY CHECKLIST

Five points — memorize this list cold

Passwords	Hashed using BcryptJS before being saved.
Authentication	Stateless JWT tokens — no session data stored on the server.
Token storage	HttpOnly, Secure cookies — prevents XSS token theft.
Environment variables	MongoDB URI, Cloudinary secret, JWT secret hidden in .env, never pushed to GitHub.
File validation	Multer strictly allows only application/pdf under 10MB, blocking malicious uploads.

10 · ERROR HANDLING & DEPLOYMENT

Q: What happens if an API request fails?

A: A global error handler in server.js catches any crash and sends a clean 500 Server Error JSON response instead of shutting down the Node process. The React frontend catches this and shows a toast notification.

Q: How did you configure environment variables for deployment?

A: Locally they live in a .env file. In production (Vercel), the same variables are entered manually into the Vercel dashboard so the deployed app can securely reach the production database and Cloudinary account. The Axios client automatically switches between localhost and the deployed URL based on environment.

11 · TRICKY PROFESSOR FOLLOW-UP QUESTIONS

Practice these dialogues out loud

Professor: “Why did you use JWT instead of standard sessions?”

You: “Because JWT helps authenticate users without storing session data on the server.”

Professor (follow-up): *“Okay, but if the JWT is stored on the client side, how do you prevent hackers from stealing it using JavaScript (XSS)?”*

Ideal answer: In my project, the JWT is NOT stored in localStorage where JavaScript can read it. My backend sends the token back as an HttpOnly cookie. This tells the browser to lock the cookie away from JavaScript entirely — it just automatically attaches it to subsequent API requests.

Professor: “Why did you use Cloudinary? Why not just save the PDF directly into MongoDB?”

You: “Because storing large files directly in the database is inefficient.”

Professor (follow-up): *“But couldn't you technically still store it as binary data?”*

Ideal answer: MongoDB has a strict 16MB document size limit. Storing PDFs as binary data would bloat the database, cause slow search queries, and quickly hit that limit. Cloudinary handles large files efficiently, acts as a CDN for fast loading, and keeps my database lightweight by only storing the URL string.

12 · TOP 30 MUST-MEMORIZE VIVA QUESTIONS

VERY HIGH PRIORITY

1. **What is the complete architecture of your project?**
   React Frontend → Express Backend → MongoDB/Cloudinary.
2. **How does JWT work in YOUR code?**
   Generated on login, stored in HttpOnly cookie, verified by authMiddleware on protected routes.
3. **How exactly does a student upload a PDF?**
   FormData → Multer tmp folder → Cloudinary upload → fs.unlinkSync deletes tmp → MongoDB saves URL.
4. **How do you secure passwords?**
   BcryptJS hashing inside a Mongoose pre-save hook.
5. **Why NoSQL/MongoDB instead of SQL?**
   Document structure matches JavaScript JSON objects, giving flexibility for different user roles.
6. **How does the force-password-change work?**
   Backend generates a random password, sets mustChangePassword: true; frontend intercepts login and routes to change-password screen.
7. **How do you prevent XSS attacks?**
   HttpOnly cookies for JWTs.
8. **What does Multer do?**
   Parses multipart/form-data so the backend can read the uploaded physical file.
9. **How do you handle backend errors?**
   Global error middleware catches exceptions and returns JSON instead of crashing the server.
10. **Explain your database schema for Assignments.**
    Stores Title, Student ObjectId, Professor ObjectId, Cloudinary URL, and Status.

HIGH PRIORITY

1. **Why React?**
   Component-based SPA for a fast, seamless experience without page reloads.
2. **What is middleware in Express?**
   Functions that run between the request and the final controller, like authMiddleware or multer.
3. **How does Nodemailer work here?**
   Connects via SMTP env credentials to dispatch welcome emails and OTPs.
4. **How are roles handled?**
   A role field in the UserData schema; backend middleware checks req.user.role === 'admin'.
5. **What is Axios?**
   An HTTP client used in React to talk to the backend, configured with withCredentials: true for cookies.
6. **How does the Professor view the PDF?**
   The Cloudinary URL is embedded into an <iframe> in the React component.
7. **What is the use of populate() in Mongoose?**
   Replaces ObjectId references with the actual related documents from another collection.
8. **Why Vite?**
   Extremely fast build tool and dev server compared to Create React App.
9. **How do you handle routing in React?**
   react-router-dom, configured in App.jsx.

10. **What happens if a user uploads a 50MB file?**
Multer is configured to reject files over 10MB, throwing a LIMIT_FILE_SIZE error gracefully.

MEDIUM PRIORITY

1. **How does Socket.io work here?**
Real-time bidirectional communication for live chat and online status.
2. **What hooks did you use in React?**
useState for form data, useEffect for fetching data on mount.
3. **What is CORS?**
Cross-Origin Resource Sharing — enabled on the backend to allow requests from the React frontend on a different port/domain.
4. **How do you store secret keys?**
In a .env file, ignored by Git.
5. **What happens on Logout?**
Backend clears the HttpOnly cookie.
6. **What is a Mongoose Model?**
A wrapper around the schema that allows querying the database.
7. **How do you handle validation?**
Frontend HTML5 validation, plus backend checks for missing fields before processing.
8. **Why didn't you use Redux?**
The app's state was simple enough to manage with local state and context/prop drilling.
9. **What was the hardest part of the project?**
Share a genuine struggle — likely the Cloudinary integration or setting up HttpOnly cookies across different domains.
10. **How did you deploy it?**
Frontend on Vercel, backend on a Node hosting service, database on MongoDB Atlas.

13 · PROJECT EXPLANATIONS

Memorize these two — they open almost every viva

2-Minute Explanation

My project is a University Assignment Portal built using the MERN stack (MongoDB, Express, React, Node.js). It solves the problem of professors receiving disorganized assignments via email. The system has three main actors: Admins who manage accounts, Students who upload PDF assignments, and Professors who review and grade them online. I used React for a fast frontend, Express/Node.js for the backend API, MongoDB to store user and assignment metadata, and Cloudinary to securely store the actual PDF files. Security is prioritized by using Bcrypt to hash passwords and JWTs stored in HttpOnly cookies for secure authentication.

5-Minute Detailed Explanation

(Start with the 2-minute intro above, then continue with:)

To walk you through a core technical workflow: when an Admin creates a new student, the backend auto-generates a secure password and uses Nodemailer to email it to the student. When the student logs in, the backend verifies the credentials using Bcrypt, and issues a JWT token hidden in an HttpOnly cookie to protect against cross-site scripting (XSS). The system detects the auto-generated password flag and forces the student to set a private password.

When the student submits an assignment, the React frontend sends the PDF via Axios using FormData. On the Express backend, Multer middleware intercepts this, temporarily saves the PDF to the server disk, checks that it is under 10MB, and then pushes it to Cloudinary. Cloudinary returns a secure URL. The backend immediately deletes the temporary local file to save server space, and saves the Cloudinary URL into the MongoDB Assignment document alongside the Student and Professor ObjectIds.

Finally, when the Professor logs in, the React frontend fetches that Cloudinary URL and displays the PDF inside an iframe, allowing the professor to read and approve the assignment without ever needing to download it. This architecture ensures the database remains fast, the server disk stays empty, and the user experience stays seamless.

Final tip: Read this handbook once fully, then revise only the orange follow-up boxes and the Top-30 list the night before. Confidence in the viva comes from explaining the *flow*, not memorizing exact code.